

# 赛题二：Matmul 算子

## 1. 赛题背景

MatMul（矩阵乘法，Matrix Multiplication, MM）是深度学习模型中最核心、最基础的计算密集型算子之一，大量应用于全连接层、卷积计算的等价变换、线性映射与 Attention 计算中。在大模型（LLM）训练与推理场景下，MatMul 的性能极大地影响模型的整体吞吐率，其优化高度依赖于硬件架构特性、数据布局以及计算与访存的协同设计。

本赛题要求选手在 **Ascend 910B3** 硬件平台上，使用 **Triton-Ascend** 或 **TileLang-Ascend** 编写 Matmul 算子。

## 2. 算子功能

完成矩阵 A 与矩阵 B 的乘法，得到矩阵 C。在 torch 的实现中可直接调用 `torch.matmul` 接口。

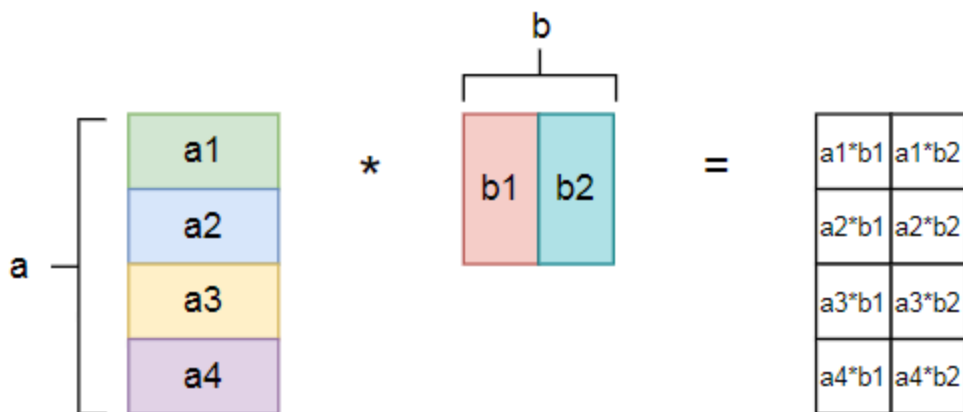
在完成 triton/tilelang kernel 时，需遵循以下规则：

- （1）每个线程只计算  $BLOCK\_SIZE\_M * BLOCK\_SIZE\_N$  大小的结果矩阵并存到 C 结果矩阵中
- （2）线程内计算 C 的时候需采用分块矩阵乘法，把 K 这个维度按  $BLOCK\_SIZE\_K$  的大小分块，用循环的方式把每一块的乘法计算结果加起来

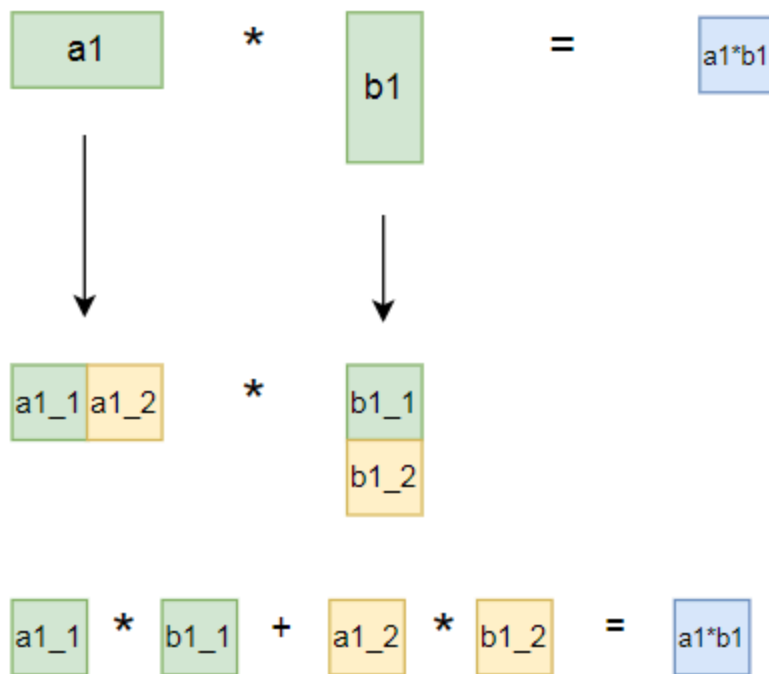
原因是：计算核心的存储空间有限，把整个矩阵一次性搬入可能导致空间不足。分块是较为标准的做法。

分块矩阵乘法思路如下图所示（实际实现时按具体的  $BLOCK\_SIZE\_M/N/K$  进行分块）：

多线程并行



单线程内分块  
(以  $a_1 \cdot b_1$  举例)



### 3. 输入输出

输入矩阵形状为 A: [512, 256] B: [256, 256], 输出矩阵形状为 C: [512, 256]。

**Triton/TileLang kernel** 的两个输入矩阵数据类型为 `float16` , 输出矩阵的数据类型为 `float32` 。

## 4. 答题要求

代码框架在平台的共享文件存储中已经给出（权限为只读），路径如下。

源路径：[/matmul](#)

挂载路径：[/2-matmul/matmul](#) 

请你在已给出的 `matmul_triton.py` 和 `matmul_tilelang.py` 中任选其一，补全其中的 **todo**，并运行文件通过结果正确性测试。请在希冀平台的对应作业中提交你的**代码文件**和**运行结果截图**。

注：你可以适当的合理修改给出的框架中 `matmul_kernel` 和 `matmul` 函数的内容。